ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

EEM-272: Model İyileştirme ve Transfer Öğrenme

Dersin Öğretim Üyesi: Dr. Öğr. Üyesi Ali ŞENTÜRK

Hazırlayan: Umut Can 2212705045

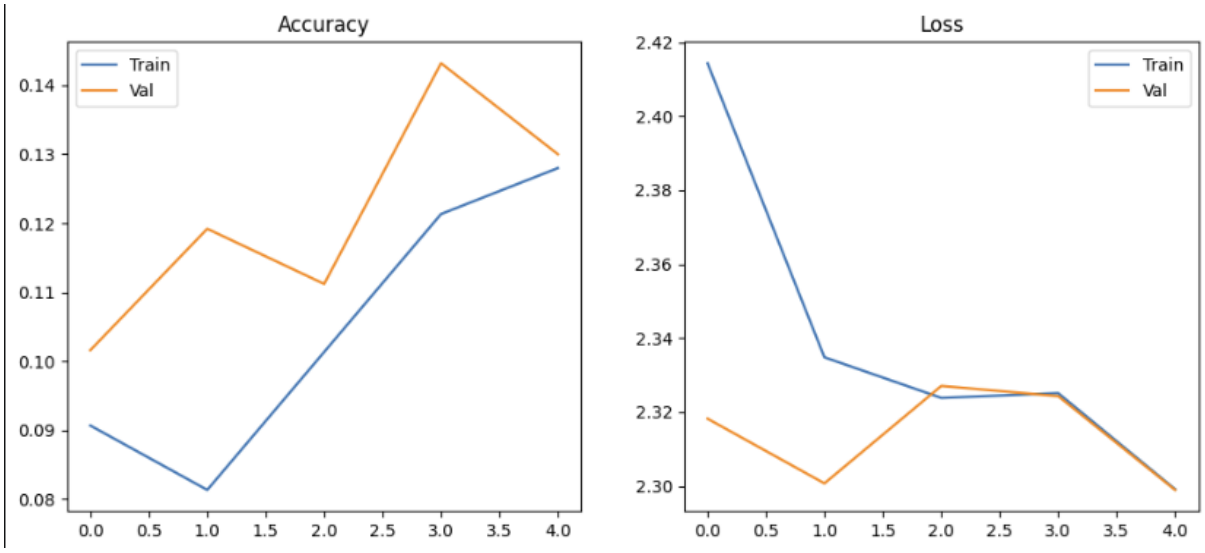
1. GİRİŞ

Günümüzde derin öğrenme modellerinin başarısı sadece karmaşık mimarilerin kurulmasına değil, bu modellerin veri setlerine nasıl adapte edildiğine ve eğitim sonrası karşılaşılan hataların nasıl yönetildiğine bağlıdır. Bir modelin eğitim setindeki başarısı, gerçek dünya verileriyle karşılaştığında "Dağılım Kayması" (Distribution Shift) gibi problemler nedeniyle düşebilmektedir.

2. HATA ANALİZİ (ERROR ANALYSIS)

Hata analizi, modelin yanlış tahminlerde bulunduğu örneklerin manuel olarak incelenmesi sürecidir. Bu süreç, hangi iyileştirme stratejisinin en düşük tahmin hatalarını sağlayacağını gösterir.

- **Kök Neden Analizi:** Eğer hataların büyük çoğunluğu bulanık görüntülerden kaynaklanıyorsa, model mimarisini değiştirmek yerine veri setine odaklanmak daha mantıklıdır.
- **Hata Tablolama:** Her bir yanlış sınıflandırma; "düşük ışık", "arka plan kalabalığı" veya "yanlış etiketleme" gibi kategorilere ayrılarak yüzdesel olarak raporlanır.



Verilerin hatalı işlendiği model çıktısı örneği yukarıdaki gibidir.

Karmaşıklık Matrisi (Confusion Matrix), makine öğrenmesinde (özellikle sınıflandırma problemlerinde) eğittiğin bir modelin performansını ölçmek ve modelin tam olarak nerede ve ne tür hatalar yaptığını görmek için kullanılan özet bir tablodur.

3. DAĞILIM KAYMASI (DISTRIBUTION SHIFT)

Eğitim verisi (Train) ile uygulama verisi (Production/Dev) arasındaki istatistiksel tutarsızlık "Dağılım Kayması" olarak adlandırılır.

- **Covariate Shift:** Modelin girdi özelliklerinin (X) dağılımındaki değişimdir. Örneğin, gündüz fotoğraflarıyla eğitilen bir otonom sürüş sisteminin gece sürüşünde zorlanması.
- **Concept Drift:** Girdi ile hedef çıktı arasındaki ilişkinin değişmesidir. Bu durum, modelin güncel verilerle periyodik olarak yeniden eğitilmesini gerektirir.
- **Label Shift:** Modelin hedef değişkeninin (y) dağılımındaki değişimdir. Örneğin, normal koşullarda çoğunlukla sağlıklı bireylerden oluşan bir veriyle eğitilen bir hastalık teşhis modelinin, salgın döneminde hasta oranının arttığı bir veri üzerinde kullanıldığında performansının düşmesi.

4. TRANSFER ÖĞRENME (TRANSFER LEARNING)

Transfer öğrenme, bir alanda (Source) eğitilmiş bir modelin bilgisini, benzer özelliklere sahip farklı bir alana (Target) aktarmaktır. Özellikle tıbbi görüntüleme gibi veri setinin kısıtlı olduğu alanlarda, ImageNet gibi devasa veri setlerinde eğitilmiş modellerin (ResNet, VGG) kullanımı standart bir uygulama haline gelmiştir. Bu yöntem, modelin temel görsel özellikleri (kenar, doku, şekil) sıfırdan öğrenmesi yerine, mevcut bilgiyi kullanmasını sağlar.

5. ÇOK GÖREVLİ ÖĞRENME (MULTI-TASK LEARNING)

Tek bir modelin aynı anda birden fazla ilişkili görevi (örneğin nesne tespiti ve segmentasyon) öğrenmesi sürecidir.

- **Avantajları:** Paylaşılan katmanlar sayesinde model, daha genel özellikler öğrenir ve aşırı öğrenme (overfitting) riski azalır.
- **Uygulama:** Kayıp fonksiyonu (Loss Function), her bir görevin hatasının ağırlıklı toplamı olarak hesaplanır.

$$L_{total} = \sum_i w_i L_i$$

6. FINE-TUNING VS FEATURE EXTRACTION

Aktarımlı öğrenmede iki temel yaklaşım mevcuttur:

- **Özellik Çıkarımı (Feature Extraction):** Önceden eğitilmiş modelin evrişimsel tabanı tamamen dondurulur. Sadece en sondaki sınıflandırıcı katman yeni verilere göre eğitilir. Veri setinin çok küçük olduğu durumlarda tercih edilir.
- **İnce Ayar (Fine-tuning):** Önceden eğitilmiş modelin son birkaç evrişimsel katmanının da ağırlıkları güncellenir. Bu, modelin yeni veri setindeki spesifik detaylara uyum sağlamasına yardımcı olur.

7. FINE-TUNING STRATEJİLERİ

Modelin performansını optimize etmek için kullanılan başlıca teknikler şunlardır:

- **Katman Dondurma (Freeze Layers):** Eğitimin başında ilk katmanlar dondurularak genel özelliklerin bozulması engellenir.
- **Kısmi Eğitim (Partial Training):** Modelin sadece belirli bloklarının (örneğin son iki blok) eğitilmesi, hesaplama maliyetini düşürürken doğruluğu artırır.

8.GÜNDELİK HAYATTA ÖRNEK KULLANIM ALANLARI

Günlük hayatta model iyileştirme ve transfer öğrenme teknikleri birçok yapay zekâ uygulamasında kullanılır. Örneğin yüz tanıma sistemleri, ImageNet gibi büyük veri setleriyle önceden eğitilmiş modellerin farklı kullanıcı verilerine uyarlanmasıyla çalışır ve böylece düşük veri ile yüksek doğruluk elde edilir.

Görüntü işleme tabanlı fotoğraf uygulamalarında, önceden eğitilmiş CNN modelleri nesne ve yüz tanıma görevlerine uyarlanır. Bu sayede sistemler, yeni veriler üzerinde yeniden sıfırdan eğitilmeden yüksek performans gösterebilir.

Sosyal medya ve içerik platformları, kullanıcı davranışlarını analiz ederek öneri sistemlerini sürekli günceller. Bu süreçte transfer öğrenme ve model fine-tuning yöntemleri kullanılarak kişiselleştirilmiş içerik üretimi sağlanır.

9. MİNİ PROJENİN ANALİZİ VE DEĞERLENDİRMESİ

```python
import os # Bilgisayardaki dosya ve klasör yollarını kontrol etmek için.
import matplotlib.pyplot as plt # Modelin başarısını çizgi grafiklerle göstermek için.
import numpy as np # Üretilen tahmin sonuçları üzerinde matematiksel/dizi işlemleri yapmak için.
import tensorflow as tf # Derin öğrenme mimarimizi kurduğumuz ana kütüphanemiz.
from tensorflow.keras import layers, models, applications # Katmanları ve önceden eğitilmiş modelleri almak için.
from sklearn.metrics import classification_report, confusion_matrix # Başarı metriklerini ve hata raporunu oluşturmak için.
import seaborn as sns # Hata matrisini renkli bir tablo (heatmap) olarak çizdirmek için.
```

```python
# TensorFlow'un sunduğu örnek çiçek veri setinin adresini belirtiyor ve arşivden çıkarıyoruz.
dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
data_dir = tf.keras.utils.get_file('flower_photos', origin=dataset_url, untar=True)

# KLASÖR DÜZELTMESİ: Arşivden çıkan dosya "flower_photos/flower_photos" gibi iç içe
# geçmişse, ana dizini doğru ayarlıyoruz ki model 5 farklı sınıfı (papatya, gül vb.) görebilsin.
if os.path.exists(os.path.join(data_dir, 'flower_photos')):
    data_dir = os.path.join(data_dir, 'flower_photos')

# Fotoğrafların yeniden boyutlandırılacağı standart ölçüyü (160x160) ve
# modelin tek seferde okuyacağı fotoğraf sayısını (32) belirliyoruz.
IMG_SIZE = (160, 160)
BATCH_SIZE = 32

# Veri setinin %80'ini modelin öğrenmesi (training) için ayırıyoruz.
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir, validation_split=0.2, subset="training", seed=123,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE)

# Veri setinin kalan %20'sini modelin kendini test etmesi (validation) için ayırıyoruz.
val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir, validation_split=0.2, subset="validation", seed=123,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE)

# Bulunan sınıf isimlerini ve toplam sınıf sayısını ekrana yazdırıyoruz.
class_names = train_ds.class_names
num_classes = len(class_names)
print(f"\nSınıf Sayısı: {num_classes} - Sınıflar: {class_names}")
```

```python
# Önceden eğitilmiş temel modeli (base_model) indiriyoruz. 'include_top=False' diyerek
# orijinal karar verme kısmını atıp kendi çiçek katmanımızı eklemek üzere hazırlarız.
base_model = applications.MobileNetV2(input_shape=IMG_SIZE + (3,),
                                      include_top=False, weights='imagenet')

# Temel modelin daha önceden öğrendiği yetenekler (kenar, renk tanıma) bozulmasın diye
# mevcut ağırlıkları donduruyoruz. Sadece kendi ekleyeceğimiz son katman eğitilecek.
base_model.trainable = False

# AKTİVASYON VE LOSS SEÇİMİ:
# Eğer 2 sınıf (kedi-köpek vb.) varsa binary (ikili) algoritmalar kullanılır.
# Eğer 5 sınıf (çiçekler gibi) varsa multiclass (çoklu) algoritmalar kullanılır.
if num_classes == 2:
    output_units = 1
    activation_fn = 'sigmoid' # İkili sınıflandırma için sigmoid
    loss_fn = 'binary_crossentropy'
else:
    output_units = num_classes
    activation_fn = 'softmax' # Çoklu sınıflandırma için softmax
    loss_fn = 'sparse_categorical_crossentropy'

# Katmanları sırayla üst üste dizerek asıl yapımızı oluşturuyoruz:
model = models.Sequential([
    layers.Rescaling(1./127.5, offset=-1), # Pikselleri MobileNetV2'nin anlayacağı aralığa (-1 ile 1) çevirir.
    base_model,                            # Yukarıda indirdiğimiz dondurulmuş ana model.
    layers.GlobalAveragePooling2D(),       # Gelen karmaşık verileri düzleştirip özetler.
    layers.Dropout(0.2),                   # Model ezberlemesin diye nöronların %20'sini rastgele uyutur.
    layers.Dense(output_units, activation=activation_fn) # Çiçeğin türüne karar veren son katman.
])

# Modelin öğrenme yöntemini (adam) ve başarısını neyle ölçeceğini (accuracy) ayarlayıp derliyoruz.
model.compile(optimizer='adam', loss=loss_fn, metrics=['accuracy'])
model.summary() # Modelin özetini (katmanları ve parametre sayısını) ekrana basar.
```

```python
# Eğitim turu (epoch) sayısı. Model tüm çiçekleri 5 kere baştan sona görecek.
epochs = 5
print("\nEğitim Başlıyor...")

# Öğrenme işlemini başlatıyoruz. Model 'train_ds' ile öğrenir, 'val_ds' ile test edilir.
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

```python
def plot_results(history, model, val_ds, class_names):
    # a) Eğitim Başarısı Grafiği
    acc = history.history['accuracy'] # Her turdaki eğitim başarısı
    val_acc = history.history['val_accuracy'] # Her turdaki test başarısı

    plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.plot(acc, label='Eğitim Başarısı')
    plt.plot(val_acc, label='Doğrulama Başarısı')
    plt.legend()
    plt.title('Doğruluk Oranları')

    # b) Tahminlerin Alınması ve Sınıflandırma Raporu
    y_true, y_pred = [], []
    for images, labels in val_ds:
        preds = model.predict(images, verbose=0) # Ayırdığımız test resimlerini modele soruyoruz.
        y_true.extend(labels.numpy())            # Gerçek cevapları kaydediyoruz.

        # Modelin verdiği yüzdelik tahminleri net sonuçlara (0, 1, 2, 3, 4) çeviriyoruz.
        if len(class_names) == 2:
            y_pred.extend((preds > 0.5).astype(int).flatten())
        else:
            y_pred.extend(np.argmax(preds, axis=1))

    # Hangi çiçek türünde ne kadar başarılı olduğumuzu ekrana yazdırır.
    print("\n--- SINIFLANDIRMA RAPORU ---")
    print(classification_report(y_true, y_pred, target_names=class_names))

    # c) Hata Matrisi (Confusion Matrix) Tablosu
    # Lale'yi Gül ile karıştırmış mı gibi spesifik hataları görmek için renkli tablo oluşturuyoruz.
    cm = confusion_matrix(y_true, y_pred)
    plt.subplot(1, 2, 2)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges', xticklabels=class_names, yticklabels=class_names)
    plt.title('Hata Matrisi')
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)

    # Grafikleri düzgün bir düzende ekranda gösteriyoruz.
    plt.tight_layout()
    plt.show()

# Yazdığımız görselleştirme fonksiyonunu çağırarak grafikleri ekrana yansıtıyoruz.
plot_results(history, model, val_ds, class_names)
```

Yapılan Mini Projenin Amacı: Bu projenin temel amacı, derin öğrenme tekniklerini kullanarak bilgisayara görüntü tanıma yeteneği kazandırmak ve fotoğrafları verilen çiçekleri türlerine (gül, lale, papatya vb.) göre otomatik olarak doğru bir şekilde sınıflandırabilen bir yapay zeka modeli geliştirmektir.

Yapılan Çalışmanın Açıklaması: Çalışma kapsamında öncelikle çiçek fotoğraflarından oluşan veri seti indirilerek eğitim ve test aşamaları için ikiye ayrılmıştır. Görüntü işleme adımları uygulandıktan sonra, önceden eğitilmiş güçlü bir yapay zeka modeli bu çiçekleri tanıyacak şekilde yeniden eğitilmiştir. Son aşamada ise modelin başarısı test edilmiş; doğruluk oranları, başarı durumu ve yaptığı tahmin hataları grafikler ve hata matrisi üzerinden görselleştirilerek analiz edilmiştir.

Kullanılan Veri Seti: Google TensorFlow'un sunduğu açık kaynaklı "Flower Photos" veri seti kullanılmıştır. Bu veri seti içerisinde 5 farklı sınıfa ait (papatya, karahindiba, gül, ayçiçeği, lale) binlerce çiçek fotoğrafı bulunmaktadır. Görüntüler modelin eğitimi için %80, performansını test etmesi için ise %20 (doğrulama seti) oranında bölünerek kullanılmıştır.

Kullanılan Yöntem: Sınıflandırma işlemi için yapay zekanın alt dalı olan derin öğrenme (deep learning) yönteminden faydalanılmıştır. Sıfırdan bir model kurmak yerine, nesne tanıma konusunda kendini kanıtlamış "MobileNetV2" sinir ağı mimarisi kullanılarak transfer öğrenimi (transfer learning) yaklaşımı benimsenmiştir. Bu sayede modelin çok daha hızlı ve yüksek doğrulukla öğrenmesi sağlanmıştır.

Kullanılan Kodlamalar (Özet): Proje Python programlama dili ile geliştirilmiştir. Kodlama sürecinde şu araçlar kullanılmıştır:

- TensorFlow & Keras: Derin öğrenme modelinin katmanlarını oluşturmak, MobileNetV2 mimarisini çekmek ve modeli eğitmek için temel olarak kullanılmıştır.
- NumPy: Modelin ürettiği sonuçlar üzerinde matematiksel hesaplamalar ve dizi (array) işlemleri yapmak için kullanılmıştır.
- Scikit-Learn (sklearn): Modelin performansını detaylı olarak ölçen sınıflandırma raporunu ve karmaşıklık (hata) matrisini oluşturmak için kullanılmıştır.
- Matplotlib & Seaborn: Öğrenme sürecindeki başarı oranlarını çizgi grafiklerle ve hata matrisini renkli bir tablo (heatmap) olarak görselleştirmek için kullanılmıştır.

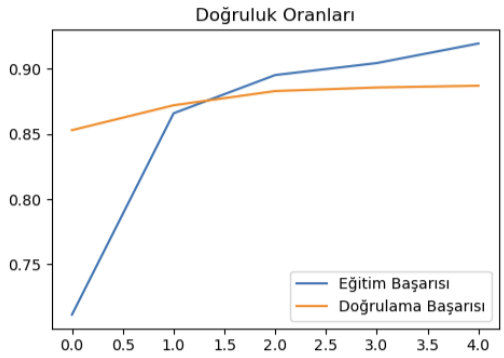
Mini Projeden Alınan Çıktılar

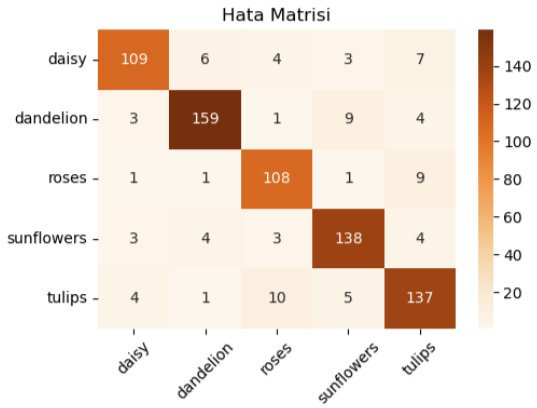
```
Eğitim Başlıyor...
Epoch 1/5
92/92 ━━━━━━━━━━━━━━━━━━━━ 18s 157ms/step - accuracy: 0.7115 - loss: 0.8011 - val_accuracy: 0.8529 - val_loss: 0.4527
Epoch 2/5
92/92 ━━━━━━━━━━━━━━━━━━━━ 11s 124ms/step - accuracy: 0.8658 - loss: 0.3918 - val_accuracy: 0.8719 - val_loss: 0.3833
Epoch 3/5
92/92 ━━━━━━━━━━━━━━━━━━━━ 12s 134ms/step - accuracy: 0.8951 - loss: 0.3088 - val_accuracy: 0.8828 - val_loss: 0.3484
Epoch 4/5
92/92 ━━━━━━━━━━━━━━━━━━━━ 12s 127ms/step - accuracy: 0.9043 - loss: 0.2669 - val_accuracy: 0.8856 - val_loss: 0.3401
Epoch 5/5
92/92 ━━━━━━━━━━━━━━━━━━━━ 11s 117ms/step - accuracy: 0.9193 - loss: 0.2268 - val_accuracy: 0.8869 - val_loss: 0.3336
```

```
--- SINIFLANDIRMA RAPORU ---
              precision    recall  f1-score   support

       daisy       0.91      0.84      0.88       129
   dandelion       0.93      0.90      0.92       176
       roses       0.86      0.90      0.88       120
  sunflowers       0.88      0.91      0.90       152
      tulips       0.85      0.87      0.86       157

    accuracy                           0.89       734
   macro avg       0.89      0.89      0.89       734
weighted avg       0.89      0.89      0.89       734
```





KAYNAKÇA:

[1] K. He, X. Zhang, S. Ren ve J. Sun, "Deep Residual Learning for Image Recognition," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770-778. [Çevrimiçi]. Erişim: https://ieeexplore.ieee.org/document/7780459

[2] S. J. Pan ve Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345-1359, Oct. 2010. [Çevrimiçi]. Erişim: https://ieeexplore.ieee.org/document/5288526

[3] X. Glorot ve Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010. [Çevrimiçi]. Erişim: https://proceedings.mlr.press/v9/glorot10a.html

[4] D. P. Kingma ve J. Ba, "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations (ICLR)*, 2015. [Çevrimiçi]. Erişim: https://arxiv.org/abs/1412.6980

[5] F. Chollet, "Deep Learning with Python," 2nd ed., Manning Publications, 2021. [Çevrimiçi]. Erişim: https://www.manning.com/books/deep-learning-with-python-second-edition

[6] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer ve N. D. Lawrence, "Dataset Shift in Machine Learning," *The MIT Press*, 2008. [Çevrimiçi]. Erişim: https://ieeexplore.ieee.org/book/6267520